T1 连续检修（repair）

我们选择两个起点，调度员选择其余设备的检修顺序。固定两个起点后，要防住的是要求最大的那一种顺序，不能只模拟某次向左或向右扩展。

先固定起点，看一台设备最晚何时被检修

设起点为 $p < q$。若第 i 台设备在第 t 步检修，就要求 $S \geq a_i + t$。因此只需确定每台设备最晚能被安排在哪一步。

- $i = p$ 或 $i = q$：它是起点，$t = 0$。
- $i < p$：在检修 i 之前，左边的 $1 \sim i-1$ 都无法到达。最多先检修 $i+1 \sim n$，其中两个起点已经完成，因此 i 最晚在第 $n-i-1$ 步检修。先把右侧全部清空，就能达到这个时刻。
- $p < i < q$：i 左右两侧各有一个起点，两侧可以分别完成检修，把 i 留到最后，所以最晚时刻为 $n-2$。
- $i > q$：与左侧对称，最晚时刻为 $i-2$。

于是固定起点的最小强度，就是所有 $a_i + t_i$ 的最大值。不同设备的最晚时刻可以由不同顺序达到：只要 S 小于其中某项，调度员便能选对应顺序让检修失败；若 S 不小于全部这些值，任何顺序都安全。

直接枚举 p, q 并扫描所有设备，需要 $O(n^3)$。预处理外侧的前缀、后缀最大值，再随 q 右移维护中间最大值，可降到 $O(n^2)$。剩下的问题是如何免去起点对的枚举。

从左到右，只需记住已经选择了几个起点

上面的罚时虽然由起点决定，却不需要知道起点的具体位置：

- 当前设备不选作起点，前面尚未选择起点，它就在两个起点左侧；
- 前面已选择一个起点，它就在两个起点之间；
- 前面已选择两个起点，它就在两个起点右侧。

设 $f_{i,j}$ 表示考虑前 i 台设备、其中恰好选择 j 个起点时，前缀所需强度最大值的最小可能值。只需 $j = 0, 1, 2$。对于相同的 i, j，后面各位置使用的规则相同；前缀要求越小越好，因此保留一个最小值足够。

考虑第 i 台设备：

1. **不选作起点。** 起点数仍为 j，罚时分别是

$$w_j(i) = \begin{cases} n-i-1, & j = 0, \\ n-2, & j = 1, \\ i-2, & j = 2. \end{cases}$$

该方案需要的强度为 $\max(f_{i-1,j}, a_i + w_j(i))$。

2. **选作起点。** 从前面已选 $j-1$ 个起点的状态转移，当前设备不扣强度，要求为 $\max(f_{i-1,j-1}, a_i)$。

在合法方案之间取最小值即可。初始 $f_{0,0} = 0$，其余状态为无穷大；$j > i$ 或剩余设备不足以补齐两个起点时，该状态无效。答案为 $f_{n,2}$。

每个位置只有三个状态，时间复杂度 $O(n)$。代码保留完整 DP 表，空间复杂度 $O(n)$。当 $n=2$ 时必须选择两台设备，答案自然为 $\max(a_1,a_2)$。最大答案不超过 10^9+n-2，使用 `int` 足够。

部分分

条件	保证分	做法与复杂度
$n \le 9$	10	枚举两个起点和所有合法扩展顺序；同一起点对取最坏要求，再在起点对间取最小。每步至多四个边界可扩展，保守时间 $O(n^3 4^n)$，空间 $O(n)$
$n \le 100$	25	用最晚检修时刻公式，枚举起点对并逐位置扫描，时间 $O(n^3)$、空间 $O(n)$
$n \le 5000$	50	左右两段预处理最大值，中间段随右起点递增维护最大值，时间 $O(n^2)$、空间 $O(n)$

这三档相互包含，分数不能相加。两个起点都必须检查自身要求；它们之间为空时，不加入中间段的罚时项。

参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 300010;
const int INF = 2000000000;
int n,a[N],f[N][3];

int main(){
    freopen("repair.in","r",stdin);
    freopen("repair.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];
    f[0][1] = INF;
    f[0][2] = INF;
    for(int i = 1; i <= n; i++){
        for(int j = 0; j <= 2; j++){
            f[i][j] = INF;
            if(j > i || 2 - j > n - i) continue;
            int t;
            if(j == 0) t = n - i - 1;
            if(j == 1) t = n - 2;
            if(j == 2) t = i - 2;
            f[i][j] = max(f[i - 1][j],a[i] + t);
            if(j > 0){
                f[i][j] = min(f[i][j],max(f[i - 1][j - 1],a[i]));
            }
        }
    }
    cout << f[n][2] << '\n';
    return 0;
}
```

T2 协同巡检（patrol）

先确定每台设备不得不走哪些边，再考虑允许停下后能省去多少返程。把树以 1 为根，根的深度为 0，记 $\mathrm{anc}_d(x)$ 为 x 向上走 d 条边后的祖先；不足 d 条边时取根。

必经子树还要包含跟随任务

若 A 必须到达 x，它一定要走过根到 x 的路径。当 A 位于 x 时，B 距离它至多为 d，所以 B 必须进入以 $\mathrm{anc}_d(x)$ 为根的子树，也就必须经过这个祖先。对 B 的任务同理。

因此，令 S_A 为 A 的原任务以及 B 原任务的 d 级祖先到根的路径并集，S_B 对称定义。它们就是两台设备的必经子树。无需反复添加跟随任务：从对方任务向上 d 步补出的点，再向上 d 步，已经在自己原有的根路径上。

这还给出一个有用性质：若 $x \in S_A$，则 $\mathrm{anc}_d(x) \in S_B$，反之亦然。于是两棵子树的交集 C 也是一棵包含根的子树，每个只属于一方的分支，从它与 C 的连接点向下最深不超过 d。两台设备可以一起遍历 C；遇到独占分支时，一台走完返回，另一台在连接点等待。由此可实现每条必经边往返一次。

记这份往返代价为

$$W = 2\big(|E(S_A)| + |E(S_B)|\big).$$

自由终点能省下多少返程

先考虑两个终点 $x \in S_A, y \in S_B$。一台设备若最终停在自己的终点，则根到终点路径上的边至少走一次，其余必经边至少走两次。能省掉的步数至多为 $\mathrm{dep}(x) + \mathrm{dep}(y)$，且终点距离必须不超过 d。令

$$M = \max_{x \in S_A,\, y \in S_B,\, \mathrm{dist}(x,y) \le d} \big(\mathrm{dep}(x) + \mathrm{dep}(y)\big).$$

终点走出自己的必经子树也不能改善这个下界。把它投影到根路径上最后一个属于自己子树的点，记两方投影为 p, q。若 p, q 互不为祖先，投影只会缩短距离。若 p 是 q 的祖先却与 q 相距超过 d，则 q 的 d 级祖先属于 S_A 且严格在 p 下方；A 的原终点不可能位于这个祖先的子树中，否则 p 不是最后一个必经点。两台设备的原终点因此也相距超过 d，矛盾。另一种祖先关系同理。因此投影后仍是一对合法候选，任何方案的代价都至少为 $W - M$。

接下来要说明，这个全局下界确实能达到。取使 M 最大的一对终点，令其最近公共祖先为 u。若 u 下方存在某个必经点 v，距 u 为 $h > d$，那么 v 与另一方必经的 $\mathrm{anc}_d(v)$ 构成合法终点对，深度和为

$$2\,\mathrm{dep}(u) + 2h - d > 2\,\mathrm{dep}(u) + d \ge M,$$

与最大性矛盾。因此，u 下方所有必经点离 u 都不超过 d。

原来的两条终点路径在 u 下方分属不同儿子分支，允许其中一条为空。在这两条分支内，设 A、B 各自必经点的最大高度为 h_A, h_B。可以让 A 最终停在高度 h_A 的点，让 B 停在另一条最深路径上高度 $b = \min(h_B, d - h_A)$ 的点。新的深度和为 $2\,\mathrm{dep}(u) + \min(d, h_A + h_B)$，不小于原来的 M，所以仍然等于 M。

先按前面的共同遍历方法完成 u 子树外的任务，最后两台设备一同到达 u。随后轮流完成各自最终分支以外的部分并返回 u；再让 B 走完自己的最终分支、停在高度 b 的位置，最后让 A 走完自己的最终分支、停在最深处。前两段移动时另一台停在 u，最后一段的距离至多为 $h_A + b \le d$。所有其他必经边恰好往返，只有两条终点路径不再返回，总代价恰为 $W - M$。

这里利用的是全局最优终点的性质，不能对任意预先指定的终点直接假定两台设备都能按各自最短路线巡检。

每棵子树只需保留两个最深深度

枚举两个终点仍需平方级时间。对每个公共必经点 u，设 $H_A(u), H_B(u)$ 为 u 子树内两方必经点的最大相对高度，则有

$$M = \max_{u \in S_A \cap S_B} \left(2\,\mathrm{dep}(u) + \min\bigl(d, H_A(u) + H_B(u)\bigr)\right).$$

对任意合法终点对，取真实的最近公共祖先，右侧对应项至少达到这对终点的深度和，所以不会漏掉最优解。

反过来，即使两个最深点来自同一个儿子，也不会高估。设选出的最深点为 p, q：若它们距离不超过 d，其深度和已经不小于该项；否则取它们真正的最近公共祖先 w，将端点沿通向 w 的路径上移，直到距离恰好为 d，所得深度和为 $2\,\mathrm{dep}(w) + d$，仍不小于该项。上移后的点仍属于各自的必经子树。因此无需区分最深点来自哪个儿子。

实现时，记 a_u, b_u 分别为 u 子树内两方**原任务点**的最大绝对深度，不存在时为 -1。A 在这棵子树内的最深必经深度为 $\max(a_u, b_u - d)$：第一项来自自己的任务，第二项来自对方任务向上 d 步后的祖先。若这个值小于 $\mathrm{dep}(u)$，则 u 不属于 S_A；否则

$$H_A(u) = \max(a_u, b_u - d) - \mathrm{dep}(u).$$

$H_B(u)$ 对称计算。按从子树到父亲的顺序合并 a, b，每个非根节点只要属于某一方的必经子树，就把对应父边的往返代价 2 加入 W；公共必经点同时更新 M。代码先用 BFS 求父亲、深度和遍历顺序，再逆序合并。总时间 $O(n)$，空间 $O(n)$。

部分分思路

下表分值表示对应点组的分值；小规模算法可以继续覆盖更小规模的点组。

条件	分值	做法
$n \le 7$	10 分	BFS 状态为两台设备的位置及两份任务的完成集合；每次只移动一台，检查距离，首次完成全部任务即为答案。直接求距离的时间上界为 $O(n^3 2^{m_A+m_B})$。
$n \le 80$	15 分	建出必经子树，枚举 $x \in S_A, y \in S_B$，沿父亲逐边求距离，按 $W - M$ 取最优，共 $O(n^3)$。
$n \le 3000$	15 分	从每个 A 候选终点 BFS 一次，得到它到所有 B 候选终点的距离，共 $O(n^2)$。逐对用倍增求 LCA 的 $O(n^2 \log n)$ 做法也可用于本档。
最大度数不超过 3	10 分	按终点真正的 LCA 分类，枚举分别供 A、B 选择的两个不同儿子分支，另计一方停在 LCA 的情况；利用分支最深高度求节省量。一般为 $O(\sum_u \deg(u)^2) = O(n\Delta)$，本档为 $O(n)$。
$d = n$	10 分	任意两点的距离均小于 d，两台设备可分别巡检。每台的代价为自己任务根路径并集的边数乘 2，再减去最深任务深度，总时间 $O(n)$。

枚举终点时应包含根和沿途节点，不能只枚举原任务点。低度树做法需要让两个非空终点分支不同；满分公式之所以允许同一儿子，是因为它对所有公共节点取最大值。d 大于树高并不保证距离限制失效，不同分支的两个点仍可能相距超过 d。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

const int N = 200000 + 10;
int n,d,fa[N],dep[N],q[N],a[N],b[N],ha[N],hb[N];
int sum,ans;
vector<int> g[N];

int main(){
    freopen("patrol.in","r",stdin);
    freopen("patrol.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> d;
    for(int i = 1; i < n; i++){
        int x,y;
        cin >> x >> y;
        g[x].push_back(y);
        g[y].push_back(x);
    }
    int l = 1,r = 1;
    q[1] = 1;
    while(l <= r){
        int x = q[l++];
        for(int i = 0; i < (int)g[x].size(); i++){
            int y = g[x][i];
            if(y == fa[x]) continue;
            fa[y] = x;
            dep[y] = dep[x] + 1;
            q[++r] = y;
        }
    }
    for(int i = 1; i <= n; i++){
        a[i] = -1;
        b[i] = -1;
    }
    int m,x;
    cin >> m;
    for(int i = 1; i <= m; i++){
        cin >> x;
        a[x] = dep[x];
    }
    cin >> m;
    for(int i = 1; i <= m; i++){
        cin >> x;
        b[x] = dep[x];
    }
    for(int i = n; i >= 1; i--){
        x = q[i];
        ha[x] = max(a[x],b[x] - d) - dep[x];
        hb[x] = max(b[x],a[x] - d) - dep[x];
        if(ha[x] < 0) ha[x] = -1;
        if(hb[x] < 0) hb[x] = -1;
        if(x != 1){
            if(ha[x] >= 0) sum += 2;
            if(hb[x] >= 0) sum += 2;
            a[fa[x]] = max(a[fa[x]],a[x]);
            b[fa[x]] = max(b[fa[x]],b[x]);
        }
        if(ha[x] >= 0 && hb[x] >= 0){
            ans = max(ans,2 * dep[x] + min(d,ha[x] + hb[x]));
        }
    }
    cout << sum - ans << '\n';
    return 0;
}
```

T3 三机编排（compile）

任务只能串行完成。安排一次加工，既决定本次费用，也会改变机器以后能否热启动；因此不能每次只选择当前费用最小的机器。

刚用过一台，另外两台必须一起记录

直接搜索每项任务交给哪台机器，有 3^n 种分配。三台机器的名字并不重要：完成第 i 项以后，刚用过的机器最后类型必为 a_i，只需记录另外两台的最后类型。

但这两个类型不能各自独立取最优。保留一种工装会占用一台空闲机器，两台机器可能保留哪些工装、为了保留它们已经花了多少时间，必须作为一个共同局面考虑。

定义 $F_i(u,v)$：完成前 i 项，刚使用的机器最后类型为 a_i，另两台最后类型分别为 u,v 时的最小累计费用。0 表示尚未使用。两台空闲机器可以交换名字，因此 $F_i(u,v)=F_i(v,u)$；实现中存一张对称矩阵。

设 $p=a_{i-1},x=a_i$。继续使用刚才的机器，需要

$$w=\begin{cases} h_x, & p=x,\\ c_x, & p\neq x. \end{cases}$$

从 $F_{i-1}(u,v)$ 出发有三种安排：

- 继续使用刚才的机器：以费用 w 更新 $F_i(u,v)$。
- 使用类型为 u 的空闲机器：支付 h_x（$u=x$）或 c_x（$u\neq x$），新空闲类型为 p,v。
- 使用类型为 v 的空闲机器：同理，新空闲类型为 u,p。

这三种安排覆盖全部选择。具有相同三种工装的历史，未来可选操作和费用都相同，较贵的历史可以舍去。令 $a_0=0$，初始 $F_0(0,0)=0$、其余为无穷大；这相当于在三台尚未使用的机器中任选一台作为“刚才的机器”。最后对 F_n 的所有格子取最小值。

直接枚举每层的 $(k+1)^2$ 个状态，时间为 $O(nk^2+n+k)$。未出现的类型不可能成为机器的历史类型，把实际出现的 C 种类型重新编号后，可降为 $O(nC^2+n+k)$。压缩编号只用于状态，费用仍由原类型查得。

换机器的全部方案，只会改动一行和一列

刚才的转移有很强的规律：继续使用原机器，会让整张矩阵同加 w；改用某台空闲机器后，原来的活动机器变为空闲，它的类型必为 p。因此，除去整张矩阵的加法，只有第 p 行和第 p 列可能变得更优。

固定换机后留下的另一种空闲类型 v。在旧状态 $F_{i-1}(u,v)$ 中枚举将被启用的机器类型 u，需要的最小费用是

$$\min\left(\min_u F_{i-1}(u,v)+c_x,\ F_{i-1}(x,v)+h_x\right).$$

第一项把 $u=x$ 也算作冷启动，但它不会造成错误：同一状态在第二项中的费用更小或相等，因为 $h_x\le c_x$。由矩阵对称性，这个枚举同时包含了启用任意一台空闲机器的情况。

直接求每列最小值仍需扫描整张矩阵。接下来利用更新只会让少数格子变小，维护这些最小值。

保存公共增量，列最小值只会下降

将上一层写成

$$F_{i-1}(u,v)=f[u][v]+add,\qquad mn[v]=\min_u f[u][v].$$

`add` 是所有格子共用的增量。继续使用原机器，只需把它增加 w，矩阵 `f` 不动。换机器后，新的公共增量是 $add+w$，所以写入第 p 行的候选应为

$$g[v]=\min\bigl(mn[v]+c_x,\ f[x][v]+h_x\bigr)-w.$$

先用上一层的 `mn` 和 `f` 算完所有 `g[v]`，再执行

$$f[p][v]=f[v][p]=\min\bigl(f[p][v],g[v]\bigr).$$

这一步必须分成两个循环：若边算边写，后面的列可能读到本次新写入的值，使同一项任务被重复用于转移。

记这一格更新后的值为 z。第 v 列只有该格发生变化；第 p 列也因对称写入多了候选 z。所有变化都是取小，因此分别执行 `mn[v]=min(mn[v],z)`、`mn[p]=min(mn[p],z)` 即可，不需要重新扫描任何一列。

初始化 `f[0][0]=mn[0]=0`，其余为无穷大，`add=0`。每轮先算完整的 `g`，再改矩阵和列最小值，最后令 `add+=w`。无穷大状态不参与运算。所有任务结束后，答案为

$$add+\min_v mn[v].$$

归纳地看，整张矩阵的加法恰好表示继续使用原机器；第 p 行、列的候选恰好表示使用任意一台空闲机器。每格仍是对应联合状态的最小费用，列最小值也随所有下降更新，所以这个实现与原动态规划完全一致。

单组初始化需要 $O(C^2)$，每项任务处理 $C+1$ 列，总时间为 $O(C^2+nC+n+k)$，空间为 $O(C^2+n+k)$。因为 $C\le n$，时间也可简写为 $O(nC+n+k)$。整文件的总时间为 $O(\sum nC+\sum n+\sum k)$；其中 $\sum nC\le 1200\sum n\le 7.2\times10^7$。

答案不超过 $n\times10^9\le6\times10^{13}$，状态、费用和公共增量均使用 `long long`。扣去公共增量后的 `f` 可以为负，它不是实际已经花掉的时间。

部分分思路

下表分值表示各算法保证覆盖的分数，不能相加。

保证条件 / 点组	分值	做法
点 $1\sim3$：$\sum n\le11$	15 分	搜索每项交给三台机器中的哪一台，维护三台机器的旧类型与累计费用，取完整分配的最小值。总时间 $O(\sum 3^n+\sum n+\sum k)$。
点 $1\sim7$	35 分	按正文的联合状态，逐层枚举全部 $(k+1)^2$ 个状态并尝试三台机器；单组 $O(nk^2+n+k)$。点 $1\sim3$ 的总 nk^2 至多 11×1200^2；点 $4\sim7$ 至多 200^3。
点 $8\sim10$：$C\le3$	15 分	三台机器各负责一种类型，首次支付冷费用，以后支付热费用，达到每项费用的下界；单组 $O(n+k)$。出现第四种类型后，这个结论不再成立。
点 $1\sim13$	65 分	仍然逐状态做三类转移，只将状态类型压缩为实际出现的 C 种；单组 $O(nC^2+n+k)$。小规模时 $C\le n$，其余相应点组满足 $C\le8$。

所有做法都要读入全部 k 项费用。类型编号范围大而实际类型数少时，压缩可以省去大量永远不可达的状态；它与维护列最小值解决的是两种不同的重复工作。

参考程序

```
#include<bits/stdc++.h>
using namespace std;

const int N = 60000 + 10;
const int K = 1200 + 10;
const long long INF = 4e18;
int n,k,a[N],id[K],type[K];
long long cold[K],hot[K],f[K][K],mn[K],g[K];

int main(){
    freopen("compile.in","r",stdin);
    freopen("compile.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while(t--){
        cin >> n >> k;
        for(int i = 1; i <= k; i++) id[i] = 0;
        int c = 0;
        for(int i = 1; i <= n; i++){
            int x;
            cin >> x;
            if(id[x] == 0){
                c++;
                id[x] = c;
                type[c] = x;
            }
            a[i] = id[x];
        }
        for(int i = 1; i <= k; i++) cin >> cold[i];
        for(int i = 1; i <= k; i++) cin >> hot[i];
        for(int j = 0; j <= c; j++){
            mn[j] = INF;
            for(int l = 0; l <= c; l++) f[j][l] = INF;
        }
        f[0][0] = 0;
        mn[0] = 0;
        a[0] = 0;
        long long add = 0;
        for(int i = 1; i <= n; i++){
            int x = a[i],p = a[i - 1],z = type[x];
            long long cost = cold[z];
            if(x == p) cost = hot[z];
            for(int j = 0; j <= c; j++){
                g[j] = INF;
                if(mn[j] == INF) continue;
                long long best = mn[j] + cold[z];
                if(f[x][j] != INF) best = min(best,f[x][j] + hot[z]);
                g[j] = best - cost;
            }
            for(int j = 0; j <= c; j++){
                long long best = min(f[p][j],g[j]);
                f[p][j] = best;
                f[j][p] = best;
                mn[j] = min(mn[j],best);
                mn[p] = min(mn[p],best);
            }
            add += cost;
        }
        long long ans = INF;
        for(int j = 0; j <= c; j++) ans = min(ans,mn[j]);
        cout << add + ans << '\n';
    }
    return 0;
}
```

T4 区间合码（orcost）

一次询问要求把区间内所有长度至少为 2 的子段各计算一次，再把这些最小合码相加。记子段代价为 $c(x,y)$，询问答案为 $S(l,r)$。

从所有子段到少量候选数对

先固定左端点 x，依次加入右端点 y。维护已有子段的最小合码，再枚举新加入的 a_y 与每个旧元素组成的数对，就能用 $O(n^3)$ 时间求出全部 $c(x,y)$。

预处理 $S(l,r)$ 时，区间 $[l+1,r]$ 与 $[l,r-1]$ 中的子段之并，恰好包含 $[l,r]$ 内除 $[l,r]$ 自身外的所有子段；它们的交集是 $[l+1,r-1]$ 中的子段。因此：

$$S(l,r)=S(l+1,r)+S(l,r-1)-S(l+1,r-1)+c(l,r).$$

长度不超过 1 时令 $S=0$。按左端点递减、右端点递增的顺序计算后，每次询问可直接查表。

为减少新元素需要比较的对象，先证明一个位运算性质：**任取 $B+1$ 个小于 2^B 的非负整数，其中存在两个数，按位或不大于这批数的最大值。**

对 B 归纳。$B=1$ 时直接成立。考察最高位：若所有数的最高位均为 0，使用低 $B-1$ 位的结论；若最高位同时有 0 和 1，且至少有两个 0，选这两个数即可。其余情形中，最高位为 1 的数至少有 B 个，去掉最高位后用归纳假设找一对，再补回最高位即可。

把一个区间的元素按数值排序。取前 31 个时，其中存在一对的合码不大于第 31 小的数；任何使用后面元素的数对，其合码至少是该元素自身。因此最优数对一定可以从前 31 小的元素中取得。不足 31 个时保留全部元素。

固定左端点扫描右端点时，维护有序的前 31 小元素和当前最小合码。每加入一个数，只将它与旧的候选元素分别取或、更新最小值，再插入有序表并保留前 31 个。每步只需 $O(B)$，可将全表预处理降至 $O(n^2B)$，其中 $B=30$。

但满分范围无法存储所有 $c(x,y)$。接下来保留的是少量**数对**，使任意区间都至少含有一对最优候选。

将所有位置按 (a_i,i) 排序，得到互不相同的排名；位置用于打破数值相同的情况。从排名最大的元素开始删除，用双向链表维护尚未删除的位置顺序。删除位置 i 前，将它与左侧最近 31 个、右侧最近 31 个尚存位置分别组成候选数对。

任意区间都能在排名最小的 31 个位置中找到最优数对。设其中排名较大的端点为 i。删除 i 时，比它排名小的位置仍然存在。两个端点之间比 i 排名小的位置至多有 29 个，所以另一个端点一定在 i 某一侧最近的 31 个尚存位置中，候选集合不会漏掉这对数。

每个位置只产生至多 62 对，总候选数为 $O(nB)$。后续只需使用这些数对。

扫描右端点，维护当前各子段的代价

把候选数对 (i,j) 写成 $i<j$，按右端点 j 分组。扫描当前右端点 r，对每个已经生效的左端点 $x<r$ 维护：

$$F_x=c(x,r).$$

区间的左端点右移时，可选数对减少，所以 $F_1,F_2,\dots,F_{r-1}$ 非降。

一对新候选 (i,r) 的合码为 w。它能被且仅能被左端点 $x\le i$ 的当前子段使用，因此带来更新：

$$F_x\leftarrow\min(F_x,w)\qquad(1\le x\le i).$$

维护非降序列中相同值构成的连续块，用 `map` 保存每块的左端点和数值。对前缀 $[1,i]$ 取最小值时，真正改变的部分必是这个前缀的一段后缀：从包含 i 的块向左检查，直到遇到值不大于 w 的块。必要时在 $i+1$ 处分裂一块，把所有降低的块删去并合并为值为 w 的一块。每个被改变的块 $[L,R]$ 产生一次区间加，增量为 $\Delta = w - $ 旧值。

一次候选更新只会新增常数个块；被删掉的块以后不会再次被删。因此全部扫描过程中的删块次数也是 $O(nB)$，并非每个数对都要遍历整个序列。

这里还要处理新生效的左端点 $r-1$。令 $v = a_{r-1}\,\mathrm{OR}\,a_r$：先将旧位置 $[1,r-2]$ 对 v 取最小值，再在位置 $r-1$ 加入初值 v，然后处理其他右端点为 r 的候选对。这样旧位置的末尾值不大于 v，非降性质始终成立。不能直接把 v 接在旧序列末尾，否则它可能比前一块更小，后续从右向左停止的条件就不再可靠。

把当前值的变化累计为所有历史右端点的贡献

当扫描到 R 时，一次查询 $[l,R]$ 的答案为：

$$S(l,R)=\sum_{x=l}^{R-1}\sum_{y=x+1}^{R} c(x,y).$$

也就是说，每个左端点需要的不是当前 F_x，而是它自生效以来的历史和。未生效的位置统一视为 0，不进入有序块集合。

假设位置区间 $[L,U]$ 在扫描到时刻 t 时增加了 Δ。到时刻 $R\ge t$，这次变化已经出现 $R-t+1$ 次，对每个位置的历史和贡献为：

$$\Delta(R-t+1)=R\Delta-\Delta(t-1).$$

因此维护两个序列：A_x 累计所有作用于位置 x 的 Δ，D_x 累计相应的 $\Delta(t-1)$。则 A_x 就是当前 F_x，位置 x 的历史和为 RA_x-D_x。

每次区间加 Δ，分别对 A 做区间加 Δ、对 D 做区间加 $\Delta(t-1)$；查询只需：

$$S(l,R)=R\sum_{x=l}^{n}A_x-\sum_{x=l}^{n}D_x.$$

尚未生效的位置仍为 0，所以可以直接求到 n。将询问按右端点分组，在该时刻的所有更新完成后回答，再按输入次序输出。

一个支持区间加、区间求和的序列可用两棵树状数组维护。若其差分为 d_i，分别维护 d_i 和 $(i-1)d_i$，则前缀和为：

$$\sum_{i=1}^{p}C_i=p\sum_{i=1}^{p}d_i-\sum_{i=1}^{p}(i-1)d_i.$$

对 A,D 各使用两棵即可。代码中的 `bit1`、`bit2` 维护 A，`bit3`、`bit4` 维护 D；`total1`、`total2` 记录两个序列的总和。`change_sum` 接收一块的变化，`change` 完成一次前缀取最小值。

候选对生成、块维护和树状数组更新合计 $O((nB+q)\log n)$ 时间，空间 $O(nB+n+q)$。

所有贡献和树状数组使用 `long long`。令 $V=2^{30}-1$，答案不超过 $n(n-1)V/2$。还需考虑树状数组内带时间和位置的系数：每个 $|D_x|\le nV$，其带位置的差分区间和可通过分部求和界为 $2n^2V$；一次区间更新的两个差分端点尚未全部加入时，再增加至多 n^2V。本题 $n\le 50000$，故保守上界 $3n^2V=8053063672500000000<2^{63}-1$，中间运算也可安全存储。

部分分思路

下表分值指互不重叠的测试点组；第二种算法同时覆盖前两组。

条件	点组分值	做法
$n \le 400$	15 分	固定左端点递增右端点，枚举新元素与所有旧元素的合码，求出全部 $c(x,y)$，再用容斥递推 $S(l,r)$；时间 $O(n^3+q)$，空间 $O(n^2)$。
$n \le 5000$	20 分	维护前 31 小元素和当前最小合码，每加入一个元素只比较 $O(B)$ 次，再预处理整张答案表；时间 $O(n^2B+q)$，空间 $O(n^2)$。
每次询问均有 $l=1$	20 分	生成稀疏候选数对并维护当前 F 的等值块。每扫到一个右端点，将当前所有 F_x 的和加入总答案，记录每个前缀的答案；不必支持任意左端点的历史区间和。时间 $O(nB\log n+q)$，空间 $O(nB+n+q)$。

前两组与前缀询问组可按输入条件合用，保证覆盖 55 分。一般查询若按左端点分组，为每个不同左端点重新做一遍扫描，也可以正确求解，但会把扫描工作重复多次。满分方法通过历史贡献公式只扫描一次。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

const int N = 50000 + 10;
const int Q = 100000 + 10;
const int E = N * 62;
int n,q,a[N],id[N],pre[N],nxt[N],head[N],to[E],ne[E],cnt;
int qhead[N],ql[Q],qn[Q],active;
long long bit1[N],bit2[N],bit3[N],bit4[N],total1,total2,ans[Q];
map<int,int> seg;

bool cmp(int x,int y){
    if(a[x] != a[y]) return a[x] < a[y];
    return x < y;
}

void add_edge(int x,int y){
    if(x > y) swap(x,y);
    to[++cnt] = x;
    ne[cnt] = head[y];
    head[y] = cnt;
}

void add(int p,long long x,long long y,long long z,long long w){
    for(int i = p; i <= n; i += i & -i){
        bit1[i] += x;
        bit2[i] += y;
        bit3[i] += z;
        bit4[i] += w;
    }
}

void change_sum(int l,int r,long long v,int t){
    long long w = v * (t - 1);
    add(l,v,v * (l - 1),w,w * (l - 1));
    add(r + 1,-v,-v * r,-w,-w * r);
    total1 += v * (r - l + 1);
    total2 += w * (r - l + 1);
}

void query(int p,long long &x,long long &y){
    long long s1 = 0,s2 = 0,s3 = 0,s4 = 0;
    for(int i = p; i > 0; i -= i & -i){
        s1 += bit1[i];
        s2 += bit2[i];
```

```
        s3 += bit3[i];
        s4 += bit4[i];
    }
    x = s1 * p - s2;
    y = s3 * p - s4;
}

void change(int p,int v,int t){
    if(p == 0) return;
    map<int,int>::iterator right = seg.upper_bound(p);
    map<int,int>::iterator it = right;
    it--;
    if(it->second <= v) return;
    if(p < active && (right == seg.end() || right->first != p + 1)){
        seg[p + 1] = it->second;
    }
    int r = p;
    while(true){
        int l = it->first;
        change_sum(l,r,(long long)v - it->second,t);
        bool first = it == seg.begin();
        map<int,int>::iterator last = it;
        if(!first) last--;
        seg.erase(it);
        if(first){
            seg[l] = v;
            break;
        }
        if(last->second <= v){
            if(last->second < v) seg[l] = v;
            break;
        }
        r = l - 1;
        it = last;
    }
}

int main(){
    freopen("orcost.in","r",stdin);
    freopen("orcost.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> q;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
        id[i] = i;
        pre[i] = i - 1;
        nxt[i] = i + 1;
    }
    for(int i = 1; i <= q; i++){
        int r;
        cin >> ql[i] >> r;
        qn[i] = qhead[r];
        qhead[r] = i;
    }
    sort(id + 1,id + n + 1,cmp);
    for(int i = n; i >= 1; i--){
        int x = id[i],y = pre[x];
        for(int j = 1; j <= 31 && y != 0; j++){
            add_edge(x,y);
            y = pre[y];
        }
        y = nxt[x];
        for(int j = 1; j <= 31 && y <= n; j++){
            add_edge(x,y);
            y = nxt[y];
        }
        nxt[pre[x]] = nxt[x];
        pre[nxt[x]] = pre[x];
    }
```

```
    for(int r = 2; r <= n; r++){
        int v = a[r - 1] | a[r];
        change(r - 2,v,r);
        active = r - 1;
        if(seg.empty() || seg.rbegin()->second != v) seg[active] = v;
        change_sum(active,active,v,r);
        for(int i = head[r]; i != 0; i = ne[i]){
            change(to[i],a[to[i]] | a[r],r);
        }
        for(int i = qhead[r]; i != 0; i = qn[i]){
            long long x,y;
            query(ql[i] - 1,x,y);
            ans[i] = r * (total1 - x) - (total2 - y);
        }
    }
    for(int i = 1; i <= q; i++) cout << ans[i] << '\n';
    return 0;
}
```